

Założenia i projekt strony internetowej do pisania wejściówek przez studentów

29.06.2026

Streszczenie

Rekomendowany system powinien być przede wszystkim **platformą dydaktyczną z funkcją wejściówek**, a nie systemem inwazyjnego nadzoru. Oznacza to architekturę, w której student loguje się przez uczelniane SSO lub konto lokalne, przegląda materiały, uruchamia wejściówkę, a system zapisuje wyniki oraz **minimalny, proporcjonalny** zakres telemetrii potrzebny do rozliczenia podejścia i ewentualnej analizy incydentów. Z punktu widzenia RODO najważniejsze są: zasada ograniczenia celu, minimalizacji danych, ograniczenia przechowywania, privacy by design/by default, właściwa podstawa prawna, przejrzysta informacja dla studentów oraz — ze względu na systematyczne monitorowanie zachowania użytkownika — przeprowadzenie DPIA przed uruchomieniem produkcyjnym. EDPB wprost wskazuje, że DPIA jest obowiązkowa lub co najmniej bardzo silnie rekomendowana przy systematycznym monitorowaniu, danych dotyczących osób w relacji zależności oraz nowych rozwiązaniach technologicznych; UODO zaleca, by w razie wątpliwości DPIA po prostu wykonać. [1]

Najważniejszy wniosek projektowy brzmi: **nie należy rejestrować surowej treści naciśniętych klawiszy ani pełnego session replay**, jeśli celem ma być ocena wejściówki i integralności podejścia. Takie podejście bardzo łatwo narusza zasadę minimalizacji i zwiększa ryzyko pozyskania danych nadmiarowych, w tym treści odpowiedzi, danych szczególnych kategorii wpisanych przypadkowo lub wzorców behawioralnych używanych potem do profilowania. Bezpieczniejszy wariant to logowanie metadanych zdarzeń: czasu, typu zdarzenia, aktywnego pytania, liczby użytych klawiszy nawigacyjnych/usuwania, zagregowanej aktywności myszy i zdarzeń widoczności karty, bez zapisu znaków wpisywanych przez studenta. To jest zgodne z logiką privacy by default i z obowiązkiem stosowania środków technicznych i organizacyjnych już na etapie projektowania. [2]

Rekomendowana architektura to **frontend Next.js/React/TypeScript**, **backend NestJS**, **PostgreSQL** jako baza główna, **Redis** do rate limitingu i krótkotrwałych kolejek/TTL, obiektowe archiwum logów w stylu S3 dla zimnych danych oraz **OIDC** do uwierzytelniania. Do ochrony danych studenta należy wdrożyć TLS, szyfrowanie w spoczynku, RBAC, audyt, separację ról, polityki retencji i eksport danych dla osoby, której dane dotyczą. Jeżeli uczelnia jest podmiotem publicznym, trzeba od początku włączyć IOD/DPO i oprzeć projekt na podstawie prawnej właściwej dla zadania pu-

blicznego lub wykonania umowy/regulaminu; zgoda może być ewentualnie używana tylko dla funkcji opcjonalnych, które nie warunkują dostępu do usługi. [3]

Zakres i założenia projektowe

Projekt dotyczy systemu WWW dla dwóch grup użytkowników: **studentów** i **prowadzących**. Student ma dostęp do materiałów zajęciowych i wejściówek; prowadzący publikuje materiały, konfiguruje pytania, przegląda wyniki oraz logi. Zakres obejmuje także rejestrację podejścia po numerze indeksu, pomiar czasu odpowiedzi, ograniczone śledzenie ruchu myszy, ograniczone logowanie aktywności klawiatury, zapis wyników oraz udostępnienie uprawnionemu prowadzącemu raportu po zakończeniu próby. To przetwarzanie ma charakter planowy, systematyczny i częściowo behawioralny, więc od początku powinno być projektowane zgodnie z art. 5, 25, 30, 32 i 35 RODO. [4]

W kilku wymiarach stan docelowy jest **nieokreślony** i trzeba to uczciwie odnotować: liczba studentów równoczesnych, dokładna częstotliwość wejściówek, wymagany SLA, dokładny okres przechowywania ocen wynikający z regulaminu studiów/archiwizacji uczelni, sposób integracji z uczelnianym SSO, katalog incydentów uznawanych za nadużycie oraz to, czy podmiotem wdrażającym będzie uczelnia publiczna czy niepubliczna. Te elementy wpływają zarówno na podstawę prawną, jak i na budżet, dobór hostingu oraz retencję. Zgodnie z EDPB privacy by design wymaga, by środki były dobierane do charakteru, zakresu, kontekstu, celu oraz ryzyka przetwarzania; bez tych danych nie da się zamknąć części decyzji projektowych „na twardo”. [5]

Przy tych nieokreślonościach najbardziej rygorystyczny i praktyczny wariant projektowy jest następujący: **MVP dla 200–500 studentów współbieżnych**, z możliwością późniejszego skalowania poziomego API i partycjonowania logów. Jest to założenie robocze, nie wymóg wynikający z dokumentów źródłowych. W całym raporcie wszystkie parametry zależne od lokalnych uchwał lub regulaminów oznaczam jako „**nieokreślone**” tam, gdzie nie da się ich ustalić bez danych od uczelni.

Wymagania funkcjonalne i нефункционалне

Cele funkcjonalne

Funkcjonalnie system powinien realizować pięć celów. Po pierwsze, **udostępniać materiały zajęciowe** w kontrolowanym dostępie dla zapisanych studentów. Po drugie, **uruchamiać wejściówki** z limitem czasu i kontrolą stanu sesji. Po trzecie, **rejestrować podejście** studenta wraz z numerem indeksu i identyfikatorem próby. Po czwarte, **zapisywać wynik i przebieg podejścia** w formie nadającej się do audytu. Po piąte, **udostępniać prowadzącemu raport** zawierający wynik, czasy per pytanie oraz zagregowane logi telemetryczne. Takie cele są zgodne z zasadą purpose li-

mitation, o ile system nie rozszerzy przetwarzania na cele wtórne, których student nie mógł racjonalnie oczekiwać. [6]

Szczegółowe wymagania dla funkcji

Funkcja	Wymaganie minimalne	Wymaganie rekomendowane	Uwagi prawne i projektowe
Rejestracja po-dejścia z numerem indeksu	Student podaje numer indeksu lub system pobiera go z SSO; system tworzy attempt_id	Numer indeksu przechowywany szyfrowany, a do wyszukiwania używany skrót/HMAC deterministyczny	Numer indeksu jest daną osobową; nie należy wymagać danych nadmiarowych
Materiały zajęć	Lista materiałów wg kursu i grupy	Wersjonowanie materiałów, publikacja/wycofanie, filtrowanie po tygodniu/zajęciach	Dostęp wyłącznie dla uprawnionych ról
Uruchomienie wejściówki	Walidacja okna czasowego, maksymalnej liczby prób, statusu użytkownika	Pin/hasło sesji, whitelisty kursów, opcjonalna integracja z harmonogramem	Warunki startu muszą być znane przed rozpoczęciem
Pomiar czasu odpowiedzi na pytanie	Rejestrowanie first_seen_at, answer_changed_at, final_answer_at, dwell_ms	Dodatkowo liczba zmian odpowiedzi i liczba powrotów do pytania	Nie wymaga danych ponadczasowych ani treści keylogów
Śledzenie ruchu myszy	Zapis próbek pozycji względnej i aktywności w zadanym interwale	Zapis tylko metryk zagregowanych: dystans, bezczynność, opuszczenia okna pytania	Zalecana próbkowana telemetria, nie pełna rekonstrukcja sesji
Logowanie naciśnięć klawiszy	Rejestracja zdarzeń key-down/keyup bez zapisu treści	Logowanie wyłącznie klas klawiszy: nawigacyjne, edycyjne, systemowe; bez znaków odpowiedzi	Surowy keylogging jest co do zasady nieproporcjonalny
Zapis wyników i logów dla prowadzącego	Wynik końcowy, odpowiedzi, czasy, zdarzenia telemetryczne	Widok surowy + zagregowany + eksport CSV/JSON/PDF	Prowadzący nie powinien widzieć danych spoza swoich kursów
Prawa osoby, której dane dotyczą	Informacja, dostęp, sprostowanie, procedura usuwania/ograniczenia	Samoobsługowy eksport danych i panel prywatności	Terminy i wyjątki wg art. 12, 15, 17, 21 RODO

Źródłowo uzasadnione są zwłaszcza trzy elementy tej tabeli. Po pierwsze, informacja o przetwarzaniu musi być zwięzła, przejrzysta i łatwo dostępna, a administrator ma ułatwiać wykonywanie praw. Po drugie, osoba ma prawo dostępu oraz — w odpowiednich warunkach — usunięcia albo sprzeciwu. Po trzecie, jeśli na podstawie telemetrii powstawałaby decyzja wywierająca istotny skutek, nie może ona opierać się wyłącznie na przetwarzaniu zautomatyzowanym bez odpowiednich zabezpieczeń i możliwości interwencji człowieka. [7]

Wymagania niefunkcjonalne

Wymagania niefunkcjonalne powinny obejmować: dostępność, bezpieczeństwo, prywatność, audytowalność, skalowalność, odporność, użyteczność i dostępność cyfrową. Dla interfejsu należy przyjąć cel **WCAG 2.2 AA** oraz poprawne użycie semantyki/ARIA tam, gdzie komponenty nie są natywne. WCAG 2.2 jest aktualnym standardem W3C określającym dostępność treści internetowych, a WAI-ARIA 1.2 jest standardem wspierającym dostępność złożonych komponentów. [8]

Dla jakości systemowej rekomenduję następujące progi projektowe: czas odpowiedzi API dla operacji typowych do 300 ms p95, zapis odpowiedzi do 500 ms p95, przyjęcie paczki telemetrii do 250 ms p95, utrata telemetrii nie większa niż 1% przy awarii pojedynczego węzła, RPO do 15 minut i RTO do 4 godzin. To są rekomendacje inżynierskie, nie progi wynikające z prawa; ich dokładna wartość pozostaje **nieokreślona** bez wymagań biznesowych uczelni.

Ramy prawne etyczne i prywatności

Podstawa prawna i zgoda

RODO wymaga wybrania właściwej podstawy prawnej z art. 6. Dla uczelni publicznej najczęściej naturalnym kandydatem będzie **zadanie realizowane w interesie publicznym / w ramach sprawowania władzy publicznej** albo przepis prawa krajowego uzupełniający ten zakres; dla uczelni niepublicznej — zależnie od modelu — możliwa jest umowa lub inna przesłanka z art. 6. Konkretna kwalifikacja jest jednak **nieokreślona** bez statusu prawnego uczelni i bez analizy regulaminów studiów. RODO wskazuje również, że dla podmiotów publicznych podstawą nie powinien być „uzasadniony interes” w zakresie wykonywania ich zadań publicznych. [9]

Zgoda nie powinna być podstawową podstawą dla obowiązkowej wejściówki lub niezbędnej telemetrii, jeżeli student nie ma realnej możliwości odmowy bez uszczerbku. EDPB podkreśla, że zgoda nie jest dobrowolna, jeżeli istnieje nierównowaga sił albo gdy usługa jest uzależniona od zgody na przetwarzanie danych, które nie są konieczne do wykonania usługi. EDPB wskazuje też przykład szkoły publicznej, w której zgoda może być ważna tylko wtedy, gdy brak zgody nie powoduje szkody dla ucznia; z tej logiki wynika, że obowiązkowa telemetria „ponad minimum” nie powinna być oparta na zgodzie. [10]

Wniosek praktyczny jest następujący. **Niezbędne przetwarzanie:** identyfikacja studenta, rejestracja próby, zapis odpowiedzi, czasy odpowiedzi, logi bezpieczeństwa i minimalna telemetria potrzebna do integralności procesu — powinny opierać się na podstawie innej niż zgoda, dobranej po analizie statusu uczelni. **Przetwarzanie opcjonalne:** analityka UX, długoterminowe zestawienia behawioralne, eksperymentalne wskaźniki „ryzyka niesamodzielnosci” — tylko jako opt-in z osobną zgodą i równoważną ścieżką bez zgody. [11]

Minimalizacja danych i retencja

Art. 5 RODO wymaga adekwatności, stosowności i ograniczenia do tego, co niezbędne, a EDPB w materiałach o privacy by design/by default wyraźnie podkreśla cztery wymiary „konieczności domyślnej”: ilość danych, zakres przetwarzania, czas przechowywania i dostępność. Domyślnie system ma więc zbierać najmniej danych, przechowywać je najkrócej, udostępniać najmniejszej liczbie osób i unikać nadmiernej granulacji. [12]

Dlatego rekomendowany model retencji jest warstwowy. **Wyniki wejściówek i odpowiedzi:** okres wynikający z regulaminu studiów, procedur odwoławczych i obowiązków archiwizacyjnych — obecnie **nieokreślone**. **Surowe logi telemetryczne:** 30–90 dni od zamknięcia okna odwoławczego. **Agregaty telemetryczne:** 180–365 dni, jeśli rzeczywiście są potrzebne do analizy jakości procesu. **Logi bezpieczeństwa i audytu administracyjnego:** 180–365 dni. **Kopie zapasowe:** 30–35 dni, z jasno opisaną procedurą odtworzenia i usuwania w cyklu backupowym. Po upływie retencji dane powinny być usuwane albo anonimizowane. EDPB wskazuje, że dane powinny być usuwane lub anonimizowane, gdy nie są już potrzebne dla celu. [13]

Dostęp, anonimizacja i pseudonimizacja

Student ma prawo do dostępu do swoich danych i informacji o celach, kategoriach danych, odbiorcach oraz czasie przechowywania; administrator powinien na takie żądania odpowiadać co do zasady w ciągu miesiąca. Jeśli student żąda usunięcia danych, administrator musi ocenić, czy nie stoją temu na przeszkodzie inne podstawy dalszego przetwarzania, np. obowiązek prawny, realizacja zadania publicznego lub obrona roszczeń. [14]

Projekt powinien rozróżniać **pseudonimizację** od **anonimizacji**. EDPB wyjaśnia, że pseudonimizacja zmniejsza możliwość przypisania danych do osoby, ale nie wyprowadza danych poza zakres RODO; anonimizacja czyni dane niepowiązawalnymi z osobą i wtedy dane przestają być danymi osobowymi. UODO zwraca uwagę, że nieskuteczna anonimizacja nie wystarcza i samo usunięcie imienia i nazwiska nie musi działać. Dlatego dla logów historycznych rekomenduję najpierw pseudonimizację operacyjną, a dopiero dla analiz długoterminowych — anonimizację nieodwracalną. [15]

DPIA, dokumentacja i etyka

Przed wdrożeniem produkcyjnym należy przygotować: rejestr czynności przetwarzania, klauzulę informacyjną, politykę retencji, umowy powierzenia z dostawcami, macierz uprawnień oraz **DPIA**. EDPB wskazuje dziewięć kryteriów pomagających ustalić potrzebę DPIA, w tym systematyczne monitorowanie, dane dotyczące podatnych grup osób i użycie nowych technologii. UODO

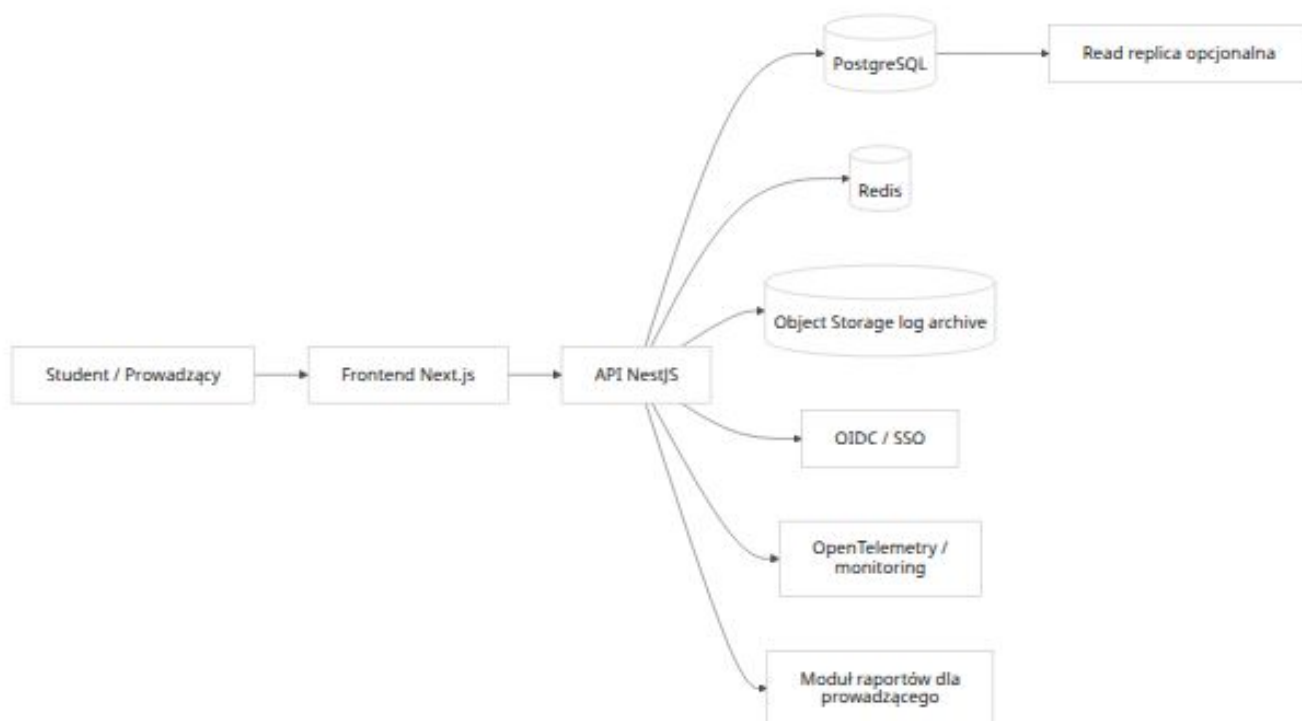
podkreśla, że w razie wątpliwości DPIA należy przeprowadzić, a jako przykład podaje systemy monitorujące zachowanie osób i przepływ informacji. [16]

Etycznie system powinien unikać „automatycznego stygmatyzowania” studentów na podstawie sygnałów behawioralnych. Telemetria może wspierać analizę incydentów, ale **nie powinna sama w sobie przesądzać o niesamodzielności**. Jeżeli zostanie użyte jakiejkolwiek punktowanie ryzyka, musi być ono ograniczone do flag pomocniczych, a każda decyzja akademicka powinna przechodzić przez człowieka. Taki wniosek wynika z art. 22 RODO oraz z zasady rzetelności i przejrzystości. [17]

Architektura technologie i bezpieczeństwo

Architektura referencyjna

Rekomenduję architekturę modułową, ale nieprzesadnie złożoną: osobny frontend, osobne API, baza relacyjna, pamięć podręczna/rate limiting, obiektowe archiwum logów i system obserwowalności. NestJS jest frameworkiem do budowy wydajnych i skalowalnych aplikacji serwerowych Node.js; Next.js zapewnia dojrzały model budowy aplikacji React z App Routerem; PostgreSQL oferuje natywne typy jsonb, partycjonowanie tabel i polityki row-level security; Redis dobrze nadaje się do TTL i limitowania rozproszonego; OpenTelemetry dostarcza standardowy model telemetrii operacyjnej. [18]



W praktyce oznacza to dwa kanały danych. **Kanał transakcyjny**: logowanie, materiały, odpowiedzi, oceny. **Kanał telemetryczny**: paczki zdarzeń wsadowych, walidowane osobno, przechowywane krócej i możliwie w formie zagregowanej. Rozdzielenie tych kanałów zmniejsza ryzyko, upraszcza retencję i poprawia wydajność.

Technologie rekomendowane

Warstwa	Technologia rekomendowana	Uzasadnienie
Frontend	Next.js + React + TypeScript	dojrzały ekosystem, App Router, dobra integracja z auth i SSR/SPA
UI	MUI lub MUI Base + własny design system	komponenty dostępnościowe i szybkie prototypowanie
Backend	NestJS	modularność, DI, kontrolery, auth/authorization
Walidacja wejścia	Zod	typowane schematy walidacji runtime
ORM / dostęp do DB	Prisma lub SQL ręczny dla logów	wysoka produktywność dla CRUD; logi dużej objętości lepiej pisać wsadowo
Baza główna	PostgreSQL	jsonb, partycjonowanie, RLS, dojrzałość
Cache / rate limiting	Redis	TTL, liczniki, rate limiter rozproszony
Uwierzytelnianie	OIDC + uczelniane SSO; alternatywnie Keycloak	standardowe endpointy, interoperacyjność
Telemetria operacyjna	OpenTelemetry	standard dla trace/metrics/logs
Telemetria kliknięć/myszy/klawiatury	natywne addEventListener, PointerEvent, KeyboardEvent, visibilitychange	najmniejsza inwazyjność i największa kontrola nad zakresem danych
Hosting	infrastruktura uczelni lub chmura UE; preferencja: region UE	prostsza zgodność, krótsza ścieżka kontraktowa i transferowa

Dobór ten opiera się na dokumentacji oficjalnej: Next.js App Router, NestJS Controllers/Providers/Auth, Zod, OpenID Connect, Keycloak, PostgreSQL jsonb/RLS/partitioning, Redis TTL/rate limiting, OpenTelemetry, MUI i W3C accessibility. [19]

Szczególnie ważna jest rekomendacja, aby **nie używać bibliotek typu pełny session replay jako domyślnego mechanizmu egzaminacyjnego**. Technicznie są możliwe, ale z perspektywy privacy by default lepszy jest własny, wąski moduł telemetryczny oparty o natywne API przeglądarki, bo pozwala precyzyjnie wykluczyć treści odpowiedzi, pola wrażliwe i niepotrzebne dane. PointerEvent standaryzuje obsługę urządzeń wskazujących, KeyboardEvent opisuje pojedyncze interakcje klawiaturą, a visibilitychange wykrywa zmianę widoczności dokumentu. [20]

Bezpieczeństwo i prywatność

W obszarze bezpieczeństwa wymagane są co najmniej: TLS dla komunikacji, szyfrowanie bazy i kopii zapasowych, bezpieczne hashowanie haseł lokalnych, centralne uwierzytelnianie, autoryzacja per rola i per kurs, dzienniki audytowe, podpisane lub wersjonowane backupy oraz segregacja dostępu „need to know”. PostgreSQL wspiera SSL/TLS oraz SCRAM-SHA-256 dla uwierzytelniania hasłowego; OWASP zaleca używanie nowoczesnych mechanizmów przechowywania haseł i zasad bezpiecznego logowania oraz kryptograficznej ochrony danych w spoczynku. [21]

Autoryzację warto realizować warstwowo. Warstwa aplikacyjna kontroluje role i uprawnienia funkcjonalne, a warstwa bazy kontroluje dostęp do rekordów. PostgreSQL RLS wprost umożliwia ograniczenie widoczności i modyfikacji wierszy przez polityki; jeśli polityki nie istnieją, działa model default deny. To jest szczególnie przydatne przy separacji kursów i prowadzących. [22]

Jeżeli korzysta się z dostawcy zewnętrznego, konieczne są umowy z podmiotem przetwarzającym zgodne z art. 28 RODO, w tym zobowiązania dotyczące poufności, bezpieczeństwa, pomocy przy realizacji praw i audytów. Jeżeli dane mają wyjść poza EOG, trzeba zastosować reżim transferowy z rozdziału V, np. mechanizm SCC. [23]

Model danych interfejs i API

Projekt bazy danych

Poniższy model zakłada zapis transakcyjny w PostgreSQL i partycjonowanie tabeli zdarzeń telemetrycznych po dacie.

Tabela	Kluczowe pola	Indeksy i ograniczenia	Cel
users	id, external_sub, email, role, status, created_at	UNIQUE(external_sub), INDEX(role,status)	tożsamość systemowa
students	user_id, student_index_enc, student_index_hmac, consent_version, consent_at	PK/FK(user_id), UNIQUE(student_index_hmac)	dane studenta i numer indeksu
courses	id, code, name, term, owner_user_id	UNIQUE(code,term)	kursy/przedmioty
enrollments	course_id, student_user_id, group_code, status	UNIQUE(course_id,student_user_id)	zapis na kurs
materials	id, course_id, title, kind, object_key, published_at, visibility	INDEX(course_id,published_at)	materiały zajęciowe
quizzes	id, course_id, title, duration_sec, available_from, available_to, max_attempts, telemetry_policy_id	INDEX(course_id,available_from,available_to)	wejściówki
questions	id, quiz_id, ordinal, type, prompt_md	UNIQUE(quiz_id,ordinal)	pytania

Tabela	Kluczowe pola	Indeksy i ograniczenia	Cel
attempts	options_jsonb, points id, quiz_id, student_user_id, attempt_no, started_at, finished_at, status, score_total, result_jsonb	UNIQUE(quiz_id,student_user_id,attempt_no), INDEX(student_user_id,started_at)	podejścia
answers	attempt_id, question_id, answer_jsonb, saved_at, finalized_at, score	UNIQUE(attempt_id,question_id)	odpowiedzi
question_metrics	attempt_id, question_id, first_seen_at, last_seen_at, dwell_ms, change_count, tab_hidden_count	UNIQUE(attempt_id,question_id)	metryki czasu i przebiegu
telemetry_events	id, attempt_id, ts, seq_no, event_type, question_id, payload_jsonb	partycjonowanie po miesiącu; INDEX(attempt_id,ts), UNIQUE(attempt_id,seq_no)	surowe zdarzenia telemetryczne
audit_log	id, actor_user_id, action, object_type, object_id, ts, result, ip_prefix, user_agent_hash	INDEX(actor_user_id,ts), INDEX(object_type,object_id)	audyt administracyjny

Ten schema wykorzystuje zalety PostgreSQL: jsonb do zdarzeń półstrukturalnych, partycjonowanie do dużych tabel logowych i RLS do ograniczania widoczności rekordów. Warto oddzielić student_index_enc od student_index_hmac: pierwsze pole służy tylko wtedy, gdy trzeba numer odszyfrować, drugie do bezpiecznego wyszukiwania i deduplikacji. [24]

Projekt interfejsu użytkownika

Rekomendowane są cztery ekrany kluczowe: logowanie, lista materiałów, wejściówka, podsumowanie wyników. Makiety poniżej są celowo proste i tekstowe; bardziej estetyczne wersje można wygenerować w Mermaid Live Editor albo odręcznie dopracować w Excalidraw. Mermaid oficjalnie udostępnia Live Editor, a Excalidraw jest narzędziem do szybkiego szkicowania diagramów i wireframe'ów. [25]

[LOGOWANIE]

```
+-----+
| Logo kursu / uczelni |
| [ Zaloguj przez SSO ] [ Konto lokalne ] |
|-----|
| Informacja o przetwarzaniu danych |
| - cele |
| - zakres telemetryczny |
| - retencja |
| [ Czytaj więcej ] |
+-----+
```

[LISTA MATERIAŁÓW]

+-----+	
Kurs: Algorytmy i Struktury Danych	
[Materiały] [Wejściówki] [Moje wyniki]	
+-----+	
Tydzień 1 PDF: złożoność obliczeniowa	
Tydzień 2 Slajdy: sortowanie	
Tydzień 3 Notatki: drzewa	
+-----+	
Najbliższa wejściówka: 2026-10-12 08:00	
+-----+	

[WEJŚCIÓWKA]

+-----+	
Pytania	Treść pytania
1 2 3 4 5 6 7 8 9 10	[timer globalny]
+-----+	
status pytania	A ()
- odwiedzone	B ()
- odpowiedziane	C ()
	D ()
+-----+	
	[Poprzednie] [Zapisz]
	[Następne] [Zakończ]
+-----+	
Pasek prywatności: logowane są metadane sesji	
+-----+	

[PODSUMOWANIE]

+-----+	
Wynik: 8/10	
Czas całkowity: 11m 32s	
+-----+	
Pytanie	wynik czas liczba zmian odpowiedzi
1	1/1 34s 1
2	0/1 91s 3
+-----+	
[Pobierz potwierdzenie] [Powrót do kursu]	
+-----+	

W interfejsie trzeba wyraźnie oddzielić część dydaktyczną od informacji o telemetrycznym monitorowaniu. Informacja o przetwarzaniu powinna być zwięzła i zrozumiała, a wybory — jeśli jakieś

są opcjonalne — nie mogą być projektowane za pomocą dark patterns. EDPB wprost wiąże privacy by design z przejrzystością, „fair choices” i łatwo dostępnymi informacjami. [26]

Przykładowe API

Endpoint	Metoda	Wejście	Odpowiedź	Uwagi
/api/auth/oidc/start	GET	—	redirect	start logowania OIDC
/api/materials?courseId=...	GET	courseId	lista materiałów	autoryzacja per kurs
/api/quizzes/{quizId}	GET	quizId	definicja wejściówki bez klucza odpowiedzi	tylko uprawniony student
/api/attempts	POST	{ quizId, stu- dentIndex } lub dane z SSO	{ attemptId, startedAt, expiresAt }	tworzy próbę
/api/attempts/ {attemptId}/answers/{qu- estionId}	PUT	{ answer }	{ ok: true, savedAt }	autosave
/api/attempts/ {attemptId}/events/batch	POST	{ seqStart, se- qEnd, events[] }	{ accepted, rejected }	wsadowa telemetria
/api/attempts/ {attemptId}/finish	POST	{ finalize: true }	{ score, summary }	zamyka próbę
/api/instructor/attempts/ {attemptId}	GET	—	wynik + metryki	rola prowadzącego
/api/instructor/attempts/ {attemptId}/logs	GET	filtry	zagregowane lub suro- we logi	dostęp limitowany
/api/privacy/export/me	GET	—	JSON/ZIP	realizacja prawa dostępu

Przykładowy schemat JSON dla paczki telemetrycznej:

```
{
  "seqStart": 120,
  "seqEnd": 134,
  "events": [
    {
      "t": "2026-10-12T08:14:12.531Z",
      "type": "pointer_sample",
      "questionId": "q3",
      "payload": { "x": 0.63, "y": 0.42, "buttons": 0 }
    },
    {
```

```

"t": "2026-10-12T08:14:15.004Z",
"type": "key_meta",
"questionId": "q3",
"payload": { "class": "editing", "code": "Backspace", "repeat": false }
},
{
"t": "2026-10-12T08:14:17.100Z",
"type": "visibility",
"payload": { "state": "hidden" }
}
]
}

```

Taki kształt API warto opisać formalnie w OpenAPI, bo OpenAPI jest standardem opisu HTTP API niezależnym od języka i ułatwia generowanie dokumentacji, klientów i testów. Walidacja wejść po stronie serwera powinna być obowiązkowa; Zod jest biblioteką typowaną do walidacji runtime i dobrze pasuje do stosu TypeScript. [27]

Krótkie przykłady kodu

Frontend TypeScript — **bez zapisu treści wpisywanych odpowiedzi:**

```

type TelemetryEvent =
  | { t: string; type: "pointer_sample"; questionId?: string; payload: { x: number; y: number; buttons: number } }
  | { t: string; type: "key_meta"; questionId?: string; payload: { kclass: "nav" | "edit" | "system"; code: string; repeat: boolean } };

const queue: TelemetryEvent[] = [];
let lastPointerSent = 0;

function currentQuestionId(): string | undefined {
  return document.querySelector<HTMLElement>("[data-question-id]")?.dataset.questionId;
}

document.addEventListener("pointermove", (e: PointerEvent) => {
  const now = performance.now();
  if (now - lastPointerSent < 250) return; // 4 Hz
  lastPointerSent = now;

  queue.push({
    t: new Date().toISOString(),

```

```

type: "pointer_sample",
questionId: currentQuestionId(),
payload: {
  x: Number((e.clientX / window.innerWidth).toFixed(3)),
  y: Number((e.clientY / window.innerHeight).toFixed(3)),
  buttons: e.buttons
}
});
}, { passive: true });

```

```

document.addEventListener("keydown", (e: KeyboardEvent) => {
  const target = e.target as HTMLElement | null;
  if (target && ["INPUT", "TEXTAREA"].includes(target.tagName)) {
    // nie zapisuj treści wpisywanej odpowiedzi
    if (!["Backspace", "Delete", "Tab", "Enter", "ArrowLeft", "ArrowRight", "ArrowUp", "ArrowDown",
"Escape"].includes(e.key)) {
      return;
    }
  }
});

```

```

const nav = ["Tab", "Enter", "ArrowLeft", "ArrowRight", "ArrowUp", "ArrowDown"];
const edit = ["Backspace", "Delete"];
const kclass = nav.includes(e.key) ? "nav" : edit.includes(e.key) ? "edit" : "system";

```

```

queue.push({
  t: new Date().toISOString(),
  type: "key_meta",
  questionId: currentQuestionId(),
  payload: { kclass, code: e.code, repeat: e.repeat }
});
});

```

```

document.addEventListener("visibilitychange", () => {
  // analogicznie: enqueue event typu "visibility"
});

```

Backend TypeScript — walidacja i zapis paczki:

```

import * as z from "zod";

const EventSchema = z.object({
  t: z.string().datetime(),
  type: z.enum(["pointer_sample", "key_meta", "visibility"]),

```

```

    questionId: z.string().optional(),
    payload: z.record(z.any())
  });

const BatchSchema = z.object({
  seqStart: z.number().int().nonnegative(),
  seqEnd: z.number().int().nonnegative(),
  events: z.array(EventSchema).max(500)
});

async function saveTelemetryBatch(attemptId: string, rawBody: unknown, userId: string) {
  const batch = BatchSchema.parse(rawBody);

  // autoryzacja: użytkownik musi być właścicielem próby albo prowadzącym danego kursu
  // kontrola spójności seqStart/seqEnd pominięta dla skrótu

  // zapis wsadowy, bez surowych znaków klawiatury
  return db.telemetry_events.insertMany(
    batch.events.map((e, i) => ({
      attempt_id: attemptId,
      seq_no: batch.seqStart + i,
      ts: e.t,
      event_type: e.type,
      question_id: e.questionId ?? null,
      payload_jsonb: e.payload,
      created_by: userId
    }))
  );
}

```

Kod celowo operuje na metadanych i walidacji schematów, a nie na „pełnym podsłuchu” aktywności. Jest to wariant bezpieczniejszy prawnie i prostszy do obrony w DPIA.

Monitoring logi wydajność koszty i wdrożenie

Mechanizmy monitorowania i logowania

Należy rozdzielić **logi operacyjne** od **logów akademickich/telemetrycznych**. Logi operacyjne służą bezpieczeństwu i utrzymaniu systemu: logowania, błędy API, audyt administracyjny, zmiany uprawnień, naruszenia limitów. Logi telemetryczne służą dokumentowaniu przebiegu podejścia: widoczność karty, zapisy odpowiedzi, metryki czasu, zagregowana aktywność mysz/klawiaturowa. OWASP zaleca budowanie mechanizmów logowania aplikacyjnego w sposób uporządkowa-

ny i bezpieczny; OpenTelemetry daje standard dla logów, metryk i śladów operacyjnych. [28] Rekomendowany format rekordu telemetrycznego:

```
{
  "attempt_id": "att_01J...",
  "seq_no": 451,
  "ts": "2026-10-12T08:14:17.100Z",
  "event_type": "visibility",
  "question_id": null,
  "payload": { "state": "hidden" },
  "schema_version": 1
}
```

Rekomendowana częstotliwość i wolumen:

- próbki myszy: 2–4 Hz,
- zdarzenia klawiatury: tylko kontrolne/nawigacyjne, bez znaków odpowiedzi,
- flush bufora: co 5–10 s albo przy 50–100 zdarzeniach,
- limit paczki: 500 zdarzeń,
- deduplikacja po attempt_id + seq_no.

Przy takich parametrach typowa paczka telemetryczna pozostaje mała, a obciążenie API przewidywalne. Dodatkowo visibilitychange pozwala wykrywać zmianę widoczności dokumentu bez cięższego monitoringu karty czy systemu operacyjnego. [29]

Wymagania wydajnościowe

Parametr	Wartość rekomendowana	Status
Czas odpowiedzi GET /materials	p95 ≤ 300 ms	rekomendacja
Czas zapisu odpowiedzi	p95 ≤ 500 ms	rekomendacja
Czas przyjęcia batcha telemetry	p95 ≤ 250 ms	rekomendacja
Utrata telemetry przy awarii pojedynczego węzła	< 1%	rekomendacja
Współbieżność	200–500 studentów	nieokreślone biznesowo
Rozmiar jednej próby telemetrycznej	~0,15–0,50 MB przy telemetry minimalnej	estymacja projektowa
Retencja surowych logów	30–90 dni	rekomendacja zgodna z minimalizacją

Największy wpływ na wydajność ma nie sama liczba studentów, lecz **to, czy logi są wysyłane pojedynczo czy wsadowo**. Wysyłanie wsadowe dramatycznie obniża liczbę requestów i koszt operacji, a przy tym ułatwia walidację i podpisywanie sekwencji zdarzeń. To jest wniosek architektoniczny, nie literalna treść źródeł.

Szacunkowe koszty

Koszty zależą od hostingu, współbieżności i retencji. Poniższa tabela pokazuje scenariusz europejski, w którym baza i API działają na infrastrukturze VM, a archiwum logów leży w obiektyowym storage. Ceny są orientacyjne, **bez VAT**, w walutach dostawców i odnoszą się do publicznie dostępnych cenników z czerwca 2026. [30]

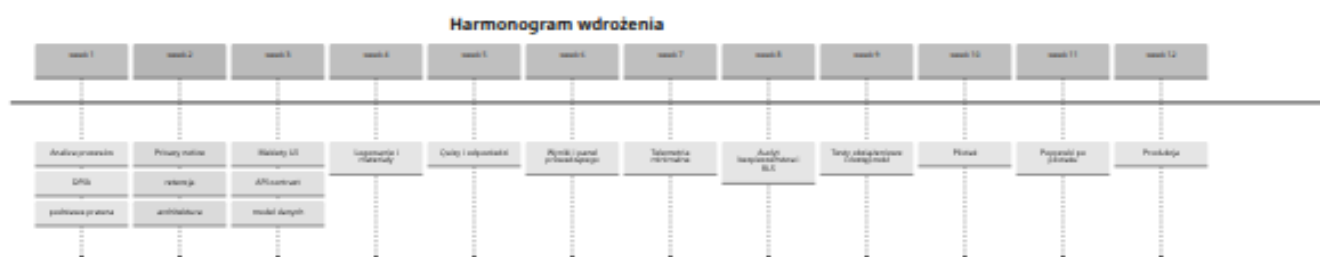
Pozycja	Scenariusz oszczędny	Scenariusz bezpieczny	Uwagi
VM aplikacyjna/API	od ok. €5.99/mies.	2× VM + LB: nieokreślone dokładnie ; rząd dziesiątek euro/mies.	koszt zależy od HA i regionu
Dodatkowy wolumen blokowy	€0.044/GB/mies.	więcej wg pojemności	dobrze dla DB i backupów
Object Storage EU	od €4.99/mies. z 1 TB storage i 1 TB egress	zwykle wystarczy dla archiwów logów	bardzo korzystne kosztowo
Cloudflare R2	\$0.015/GB-mies., free tier 10 GB; operacje wg klasy	podobnie	dobrze dla archiwum, nie dla gorącej bazy
Supabase Pro	\$25/mies.	więcej przy wielu projektach/egress	wygodne, ale wymaga analizy DPA/regionów
OVH Managed PostgreSQL	od \$64.24/mies./node	produkcyjnie od \$81.76/mies.	wygodniejsza administracja, wyższy koszt
Koszt archiwum 10 000 prób × 0,5 MB	zwykle poniżej progu bezpłatnego R2	podobnie	sama przestrzeń na logi jest tania
Koszt archiwum 100 000 prób × 0,5 MB	ok. 49 GB; storage rzędu <\$1/mies. w R2	podobnie	rosną raczej operacje i backupy niż sam storage

Najbardziej racjonalny wariant dla uczelni to zwykle **self-host lub VM w UE + PostgreSQL + tanie obiektywne archiwum**, bo ogranicza ryzyko transferowe, daje pełniejszą kontrolę nad retencją i zwykle jest tańszy niż managed DBaaS. Z kolei Supabase/managed DB może przyspieszyć MVP, ale musi zostać poprzedzony oceną regionów, DPA, listy podprocesorów i zasad transferu. Supabase udostępnia DPA, a art. 28 RODO wymaga umowy z podmiotem przetwarzającym. [31]

Plan wdrożenia i harmonogram

Kamień milowy	Zakres	Rezultat	Czas
Analiza i DPIA	mapa procesów, podstawa prawna, privacy notice, DPIA, model retencji	decyzje compliance zatwierdzone przez IOD/DPO	2–3 tyg.
Projekt UX i architektury	wireframe'y, API contract, model danych, RBAC	specyfikacja techniczna	2 tyg.
MVP dydaktyczne	logowanie, materiały, quizy, zapisy odpowiedzi, wyniki	działające środowisko testowe	4–6 tyg.

Kamień milowy	Zakres	Rezultat	Czas
Telemetria minimalna	czasy pytań, visibility, mysz 2–4 Hz, klawisze kontrolne	moduł telemetryczny v1	2 tyg.
Panel prowadzącego	raporty, eksport, filtrowanie prób, widok logów	konsola dydaktyczna	2–3 tyg.
Bezpieczeństwo i testy	testy obciążeniowe, RLS, kopie, audyt logów, testy dostępności	release candidate	2–3 tyg.
Pilotaż	1–2 grupy, analiza feedbacku i ryzyk	raport z pilotażu	2 tyg.
Produkcja	onboarding kursów, monitoring, procedury wsparcia	start semestralny	1 tydz.



Realny harmonogram zależy od integracji z SSO, gotowości infrastruktury, udziału IOD/DPO i akceptacji regulaminowej, więc daty kalendarzowe pozostają **nieokreślone**.

Ryzyka i ograniczenia

Główne ryzyka prawne i techniczne

Ryzyko	Prawdopodobieństwo	Wpływ	Mitigacja
Zbyt szeroki keylogging lub session replay	wysokie	bardzo wysokie	ograniczyć do metadanych; brak znaków odpowiedzi; DPIA
Błędna podstawa prawna	średnie	bardzo wysokie	analiza statusu uczelni i celu; zatwierdzenie przez IOD/DPO
Niewłaściwa zgoda „warunkująca” usługę	średnie	wysokie	nie opierać obowiązkowej funkcji na zgodzie
Nadmierna retencja logów	wysokie	wysokie	polityka retencji i automatyczne kasowanie
Nadmierny dostęp prowadzących	średnie	wysokie	RBAC + RLS + audyt dostępu
Transfer danych poza EOG bez zabezpieczeń	średnie	wysokie	region UE, SCC, przegląd podprocesorów
Automatyczne kwalifikowanie „oszustwa”	średnie	bardzo wysokie	tylko flagi pomocnicze, obowiązkowy review przez człowieka

Ryzyko	Prawdopodobieństwo	Wpływ	Mitigacja
Przeciążenie API przez telemetrię	średnie	średnie	batching, limity, Redis, kolejki
Nieskuteczna anonimizacja archiwów	średnie	wysokie	procedury anonimizacji, test reidentyfikacji
Braki w dostępności	średnie	średnie	WCAG 2.2 AA, testy klawiaturowe/screen reader

Z punktu widzenia prawa największe ryzyko dotyczy **proporcjonalności monitoringu**. Sam fakt, że technicznie można śledzić mysz i klawiaturę, nie oznacza jeszcze zgodności z RODO. Zasady minimalizacji, storage limitation i privacy by default muszą być materialnie odzwierciedlone w architekturze. Z punktu widzenia technicznego najważniejsze są: wsadowy zapis logów, separacja danych gorących i archiwalnych, ograniczenie odczytu surowych zdarzeń oraz możliwość szybkiego wycięcia telemetrii nadmiarowej bez przebudowy całego systemu. [32]

Pytania otwarte i ograniczenia

Nie da się obecnie rozstrzygnąć kilku kwestii bez danych lokalnych. **Nieokreślone** pozostają: dokładna podstawa prawna dla uczelni konkretnego typu, okres archiwizacji wyników wynikający z regulaminu studiów, sposób integracji z SSO, oczekiwana skala współbieżności, wymogi formalne co do odwołań od ocen oraz polityka backupów obowiązująca w danej jednostce. To nie podważa głównych wniosków raportu, ale oznacza, że przed wdrożeniem potrzebna jest krótka faza doprecyzowania organizacyjnego.

Końcowa rekomendacja jest jednoznaczna: **wdrażać system jako platformę dydaktyczną z minimalną telemetrią, nie jako narzędzie pełnego nadzoru**. Prawnie i technicznie najlepszy kompromis to: rejestracja próby, wyniki, czasy per pytanie, sygnały widoczności sesji, ograniczona i próbkowana aktywność myszy oraz wyłącznie metadane klawiatury — przy pełnym DPIA, jasnej informacji dla studentów, silnej autoryzacji i krótkiej retencji surowych logów. [33]

Zasady projektowania strony internetowej i prawidłowego tworzenia kodu WEB

Implementacja platformy do udostępniania materiałów dydaktycznych i przeprowadzania wejśćówek powinna być oparta na standardach sieciowych, zasadach inżynierii oprogramowania oraz podejściu *security and privacy by design*. Kod nie może być traktowany wyłącznie jako warstwa realizująca wymagane funkcje. Powinien być również czytelny, testowalny, odporny na błędy, dostępny cyfrowo i możliwy do dalszego rozwijania bez naruszania istniejących mechanizmów oceniania.

Semantyczna struktura dokumentu HTML

Strona powinna wykorzystywać HTML5 zgodnie z przeznaczeniem poszczególnych znaczników. Układ aplikacji należy tworzyć za pomocą elementów takich jak `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<form>` i `<footer>`, zamiast budowania całej struktury z neutralnych elementów `<div>`. Nagłówki powinny zachowywać prawidłową hierarchię: pojedynczy nagłówek `<h1>` określa główny temat widoku, a kolejne poziomy `<h2>`–`<h6>` opisują logiczne podsekcje.

Każde pole formularza musi posiadać powiązaną etykietę `<label>`. Grupy odpowiedzi jednokrotnego i wielokrotnego wyboru powinny być organizowane za pomocą `<fieldset>` i `<legend>`. Do obsługi działań należy używać przycisków `<button>`, a nie elementów `<div>` wyposażonych w obsługę kliknięcia. Zapewnia to prawidłową obsługę klawiatury, czytników ekranowych i mechanizmów nawigacyjnych przeglądarki. Przykładowa struktura pytania:

```
<fieldset class="question" aria-describedby="question-status">
<legend>
Pytanie 3. Który protokół służy do uwierzytelniania użytkownika?
</legend>

<label>
<input type="radio" name="question-3" value="a">
OAuth 2.0
</label>

<label>
<input type="radio" name="question-3" value="b">
OpenID Connect
</label>

<p id="question-status" aria-live="polite">
Odpowiedź nie została jeszcze zapisana.
</p>

<button type="button">Zapisz odpowiedź</button>
</fieldset>
```

Rozdzielenie struktury, wyglądu i logiki

Kod powinien zachowywać rozdział odpowiedzialności:

- HTML lub JSX opisuje strukturę i znaczenie treści.
- CSS odpowiada za wygląd, układ oraz responsywność.
- TypeScript realizuje logikę aplikacji, komunikację z API i obsługę zdarzeń.

- Backend odpowiada za reguły biznesowe, autoryzację, ocenianie i trwały zapis danych.
- Baza danych odpowiada za integralność i relacje między rekordami.

Nie należy umieszczać rozbudowanych stylów bezpośrednio w atrybucie `style`, logiki biznesowej w komponentach prezentacyjnych ani zapytań do bazy danych bezpośrednio w kontrolerach API. Rozdzielenie warstw ogranicza sprzężenie kodu i umożliwia niezależne testowanie poszczególnych modułów.

W omawianym systemie rekomendowany podział obejmuje moduły:

frontend/

- app/
- components/
- features/
 - materials/
 - quizzes/
 - attempts/
 - telemetry/
 - results/
- services/
- schemas/
- hooks/
- utils/

backend/

- modules/
 - auth/
 - courses/
 - materials/
 - quizzes/
 - attempts/
 - telemetry/
 - reports/
 - audit/
- common/
 - guards/
 - interceptors/
 - filters/
 - validation/

Projektowanie komponentowe

Interfejs powinien być budowany z małych komponentów o jednoznacznej odpowiedzialności. Przykładowo komponent QuizPage nie powinien samodzielnie realizować odmierzenia czasu, zapisu odpowiedzi, monitorowania aktywności, nawigacji i wyświetlania komunikatów. Funkcje te należy rozdzielić na komponenty i wyspecjalizowane hooki:

```
QuizPage
├── QuizHeader
│   ├── GlobalTimer
│   └── ConnectionStatus
├── QuestionNavigation
├── QuestionPanel
│   ├── QuestionContent
│   ├── AnswerControl
│   └── SaveStatus
├── PrivacyIndicator
└── FinishAttemptDialog
```

Logikę niezależną od prezentacji można wydzielić do hooków:

```
const timer = useAttemptTimer(expiresAt);
const autosave = useAnswerAutosave(attemptId);
const telemetry = useTelemetryCollector(attemptId, telemetryPolicy);
```

Komponent powinien otrzymywać dane poprzez jawnie zdefiniowane właściwości i emitować zdarzenia za pomocą funkcji zwrotnych. Należy unikać komponentów mających dziesiątki parametrów, globalnego stanu używanego bez uzasadnienia i powielania tej samej logiki w wielu miejscach.

TypeScript i kontrola typów

Kod frontendowy i backendowy powinien wykorzystywać TypeScript w trybie ścisłym. Należy włączyć między innymi:

```
{
  "compilerOptions": {
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true
  }
}
```

```
}
```

Nie należy nadużywać typu `any`, ponieważ wyłącza on kontrolę typów. Dane pochodzące z API, pamięci przeglądarki, parametrów adresu URL lub formularzy powinny być początkowo traktowane jako `unknown`, a następnie walidowane.

```
type AttemptStatus =  
  | "created"  
  | "in_progress"  
  | "submitted"  
  | "expired"  
  | "cancelled";  
  
interface QuestionMetrics {  
  questionId: string;  
  firstSeenAt: string;  
  dwellTimeMs: number;  
  answerChangeCount: number;  
  visibilityLossCount: number;  
}
```

Należy preferować typy opisujące dziedzinę problemową zamiast ogólnych obiektów i ciągów znaków. Utrudnia to przypadkowe przekazanie identyfikatora pytania w miejsce identyfikatora podejścia.

Walidacja danych po obu stronach aplikacji

Walidacja w przeglądarce poprawia wygodę użytkowania, lecz nie zapewnia bezpieczeństwa. Każde żądanie musi zostać ponownie zweryfikowane przez serwer. Student może zmodyfikować kod JavaScript, wysłać własne żądanie HTTP albo zmienić wartości przechowywane w pamięci przeglądarki.

Walidacji wymagają między innymi:

- format numeru indeksu;
- identyfikatory kursu, wejściówki, pytania i podejścia;
- dopuszczalny typ odpowiedzi;
- maksymalny rozmiar odpowiedzi tekstowej;
- zakres współrzędnych telemetrycznych;
- kolejność numerów sekwencyjnych zdarzeń;
- liczba zdarzeń w jednej paczce;
- znaczniki czasu;
- aktualny status podejścia;

- uprawnienie użytkownika do wskazanego zasobu.

Przykład walidacji wejścia:

```
import { z } from "zod";

const StudentIndexSchema = z
  .string()
  .trim()
  .regex(/^[0-9]{5,8}$/, "Nieprawidłowy numer indeksu");

const SaveAnswerSchema = z.object({
  questionId: z.string().uuid(),
  answer: z.discriminatedUnion("type", [
    z.object({
      type: z.literal("single_choice"),
      optionId: z.string().uuid()
    }),
    z.object({
      type: z.literal("multiple_choice"),
      optionIds: z.array(z.string().uuid()).max(20)
    }),
    z.object({
      type: z.literal("text"),
      value: z.string().max(5000)
    })
  ]),
  clientRevision: z.number().int().nonnegative()
});
```

Serwer jako źródło prawdy

Stan podejścia nie może być kontrolowany wyłącznie przez frontend. Serwer powinien rozstrzygać:

- czy wejściówka jest obecnie dostępna;
- ile podejść wykorzystał student;
- kiedy rozpoczęło się podejście;
- kiedy upływa limit czasu;
- czy odpowiedź może zostać jeszcze zmieniona;
- czy podejście zostało już zakończone;
- jaka wersja pytania została przedstawiona;
- ile punktów przyznano za odpowiedź.

Czas pozostały może być wizualizowany po stronie klienta, ale powinien wynikać ze znacznika `expiresAt` wygenerowanego przez serwer. Nie należy ufać zegarowi urządzenia studenta ani licznikowi JavaScript.

```
const remainingMs = Math.max(
  0,
  new Date(expiresAt).getTime() - Date.now()
);
```

Po wygaśnięciu czasu serwer powinien odrzucać kolejne modyfikacje, nawet jeśli przeglądarka nadal wyświetla aktywny formularz.

Bezpieczne zarządzanie stanem i zapisem odpowiedzi

Odpowiedzi należy zapisywać automatycznie po zmianie, z zastosowaniem mechanizmu opóźniającego nadmierną liczbę żądań. Interfejs musi informować o stanie zapisu: „zapisywanie”, „zapisano” lub „błąd zapisu”. Nie wolno przedstawiać odpowiedzi jako zapisanej, zanim serwer nie potwierdzi operacji.

Każda zmiana powinna posiadać numer rewizji. Pozwala to zapobiegać nadpisaniu nowszej odpowiedzi przez starsze żądanie, które dotarło do serwera z opóźnieniem.

```
interface SaveAnswerCommand {
  attemptId: string;
  questionId: string;
  answer: AnswerValue;
  clientRevision: number;
}
```

Serwer powinien wykonać operację atomowo:

```
UPDATE answers
SET
  answer_jsonb = $1,
  revision = revision + 1,
  saved_at = NOW()
WHERE attempt_id = $2
AND question_id = $3
AND revision = $4;
```

Brak zmodyfikowanego rekordu oznacza konflikt wersji, który należy rozwiązać jawnie, zamiast bezwarunkowo nadpisywać dane.

Obsługa błędów i odporność na problemy sieciowe

Aplikacja powinna zakładać możliwość chwilowej utraty połączenia, zamknięcia karty, odświeżenia strony lub błędu API. Niezapisane odpowiedzi można tymczasowo utrzymywać w pamięci lokalnej, lecz nie należy przechowywać tam danych wrażliwych ani tokenów uwierzytelniających.

Po przywróceniu połączenia aplikacja powinna ponowić zapis odpowiedzi z zachowaniem idempotencji. Endpoint nie może wielokrotnie naliczać tej samej operacji wskutek ponowienia żądania.

```
PUT /api/attempts/{attemptId}/answers/{questionId}
Idempotency-Key: 7f3d6b9f-...
```

Błędy należy dzielić na:

- błędy walidacji — kod 400;
- brak uwierzytelnienia — 401;
- brak uprawnienia — 403;
- brak zasobu — 404;
- konflikt wersji lub stanu — 409;
- przekroczenie limitu — 429;
- błąd wewnętrzny — 500.

Frontend powinien prezentować użytkownikowi komunikat zrozumiały, natomiast szczegóły techniczne należy zapisywać wyłącznie w logach serwera.

Zasady bezpieczeństwa aplikacji WEB

System powinien być projektowany zgodnie z aktualnymi zaleceniami OWASP. Wszystkie dane pochodzące od użytkownika należy traktować jako niezaufane. Podstawowe wymagania obejmują:

- wyłączone używanie HTTPS;
- bezpieczne ciasteczka z flagami HttpOnly, Secure i SameSite;
- ochronę przed CSRF;
- kodowanie danych wyjściowych i ochronę przed XSS;
- zapytania parametryzowane chroniące przed SQL Injection;
- restrykcyjną politykę Content Security Policy;
- ograniczenie liczby żądań;
- kontrolę rozmiaru przesyłanych danych;
- autoryzację sprawdzaną przy każdym żądaniu;
- unikanie ujawniania kluczy poprawnych odpowiedzi w kodzie klienta;
- rejestrowanie operacji administracyjnych;

- regularne aktualizowanie zależności.

Przykładowe nagłówki bezpieczeństwa:

```
Content-Security-Policy:
default-src 'self';
script-src 'self';
style-src 'self';
img-src 'self' data:;
connect-src 'self' https://api.example.edu.pl;
frame-ancestors 'none';

X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=(), microphone=(), geolocation=()
```

Poprawne odpowiedzi i algorytm punktowania powinny być dostępne wyłącznie po stronie serwera. Przesłanie klucza odpowiedzi w obiekcie JSON i ukrycie go za pomocą CSS nie stanowi zabezpieczenia.

Uwierzytelnianie i autoryzacja

Preferowaną metodą jest uczelniane SSO oparte na OpenID Connect. Numer indeksu powinien być pobierany z wiarygodnego profilu użytkownika, a ręczne wpisywanie stosowane wyłącznie w kontrolowanych scenariuszach zastępczych. Role powinny być jednoznaczne:

```
type Role =
  | "student"
  | "instructor"
  | "course_admin"
  | "system_admin"
  | "auditor";
```

Autoryzacja musi uwzględniać nie tylko rolę, ale także relację użytkownika z konkretnym kursem. Prowadzący przedmiot A nie powinien automatycznie uzyskiwać dostępu do wyników przedmiotu B. Administrator techniczny nie powinien domyślnie otrzymywać dostępu do merytorycznej treści odpowiedzi studentów.

Ograniczony i proporcjonalny monitoring

Kod monitorujący zachowanie studenta powinien realizować wyłącznie jawnie określone cele. Nie należy tworzyć keyloggera zapisującego wprowadzone znaki ani pełnej rekonstrukcji sesji. Zalecany zakres obejmuje:

- czas wyświetlenia pytania;

- czas aktywnego przebywania na pytaniu;
- liczbę zmian odpowiedzi;
- zmianę widoczności karty;
- próbkowaną aktywność urządzenia wskazującego;
- kategorie klawiszy nawigacyjnych i edycyjnych;
- błędy zapisu i utratę połączenia.

Zdarzenia o dużej częstotliwości, np. `pointermove`, muszą być próbkowane lub agregowane. Rejestrowanie każdego ruchu generuje nadmierny wolumen danych, obciąża interfejs i nie jest zazwyczaj konieczne.

```
const POINTER_INTERVAL_MS = 250;

let lastPointerSample = 0;

function handlePointerMove(event: PointerEvent): void {
  const now = performance.now();

  if (now - lastPointerSample < POINTER_INTERVAL_MS) {
    return;
  }

  lastPointerSample = now;

  enqueueTelemetry({
    type: "pointer_sample",
    monotonicTimeMs: now,
    payload: {
      x: event.clientX / window.innerWidth,
      y: event.clientY / window.innerHeight,
      buttons: event.buttons
    }
  });
}
```

Nasłuchiwanie należy usuwać po zakończeniu komponentu, aby zapobiegać wyciekom pamięci i wielokrotnemu rejestrowaniu tych samych zdarzeń.

```
useEffect(() => {
  document.addEventListener("pointermove", handlePointerMove, {
    passive: true
  });

  return () => {
```

```
document.removeEventListener("pointermove", handlePointerMove);  
};  
}, []);
```

Dostępność cyfrowa

System powinien spełniać wymagania WCAG 2.2 na poziomie AA. Ma to szczególne znaczenie w środowisku akademickim, ponieważ wejściówka nie może dyskryminować studentów korzystających z klawiatury, czytnika ekranowego, powiększenia, alternatywnych urządzeń wejściowych lub dostosowań wynikających z niepełnosprawności. Należy zapewnić:

- pełną obsługę bez użycia myszy;
- widoczny fokus klawiatury;
- logiczną kolejność przechodzenia klawiszem Tab;
- odpowiedni kontrast;
- tekstowe etykiety ikon;
- komunikaty błędów powiązane z polami;
- brak informacji przekazywanej wyłącznie kolorem;
- możliwość powiększenia interfejsu;
- odpowiednio duże cele dotykowe;
- brak automatycznego przesuwania fokusu bez uzasadnienia;
- komunikowanie dynamicznych zmian przez aria-live;
- mechanizm przedłużania czasu dla uprawnionych studentów.

Monitoring nie może traktować korzystania wyłącznie z klawiatury jako zachowania podejrzanego. Telemetria musi uwzględniać różne sposoby obsługi interfejsu.

Responsywność i spójność interfejsu

Układ powinien prawidłowo działać na komputerach, tabletach i urządzeniach mobilnych, choć prowadzący może określić minimalne wymagania sprzętowe dla wejściówek. Należy stosować podejście *mobile first*, elastyczne siatki i względne jednostki wymiarów.

```
.quiz-layout {  
  display: grid;  
  grid-template-columns: minmax(12rem, 18rem) minmax(0, 1fr);  
  gap: 1.5rem;  
}  
  
@media (max-width: 48rem) {  
  .quiz-layout {  
    grid-template-columns: 1fr;
```

```
}  
}
```

Nie należy ustalać sztywnej wysokości panelu z pytaniem, ponieważ dłuższa treść może zostać obcięta. Interfejs powinien zachować stabilny układ i nie przesuwać przycisków w trakcie ładowania treści.

Wydajność kodu

Wydajność ma znaczenie zwłaszcza wtedy, gdy setki studentów rozpoczynają wejściówkę jednocześnie. Należy ograniczać rozmiar pakietów JavaScript, liczbę zapytań, częstotliwość renderowania i liczbę zdarzeń telemetrycznych. Rekomendowane działania obejmują:

- dzielenie kodu na moduły ładowane na żądanie;
- kompresję zasobów;
- optymalizację plików graficznych;
- buforowanie materiałów statycznych;
- stronicowanie list wyników;
- wysyłanie telemetry w paczkach;
- indeksowanie zapytań bazodanowych;
- unikanie synchronicznych operacji wejścia-wyjścia;
- używanie `debounce` dla zapisu odpowiedzi;
- używanie `throttle` dla ruchów wskaźnika;
- ograniczanie niepotrzebnych renderów komponentów.

Telemetria nie powinna być wysyłana pojedynczym żądaniem dla każdego zdarzenia. Lepszy jest bufor opróżniany okresowo:

```
const MAX_BATCH_SIZE = 100;  
const FLUSH_INTERVAL_MS = 5000;  
  
setInterval(() => {  
  if (queue.length > 0) {  
    void flushTelemetry();  
  }  
}, FLUSH_INTERVAL_MS);
```

Czytelność, styl i dokumentowanie kodu

Kod powinien mieć spójne nazewnictwo, formatowanie i strukturę. Nazwy muszą opisywać znaczenie biznesowe:

```
// Poprawnie
```

```
calculateQuestionDwellTime();
finalizeAttempt();
isAttemptExpired;

// Niepoprawnie
calc();
dolt();
flag1;
```

Komentarze powinny wyjaśniać przyczyny decyzji, nietypowe ograniczenia i reguły biznesowe. Nie powinny powtarzać literalnie treści kodu.

```
// Zegar klienta służy wyłącznie do wizualizacji.
// Serwer weryfikuje termin wygaśnięcia przy każdym zapisie.
const remainingTime = calculateRemainingTime(expiresAt);
```

Projekt powinien stosować automatyczne formatowanie i analizę statyczną, np. ESLint, Prettier i reguły TypeScript. W repozytorium należy utrzymywać dokumentację uruchomienia, konfiguracji środowiska, migracji bazy, procedury testowania oraz opis zmiennych środowiskowych.

Zarządzanie konfiguracją i danymi poufnymi

Hasła, klucze API, sekrety sesji oraz dane dostępowe do bazy nie mogą znajdować się w kodzie źródłowym ani w repozytorium. Należy przekazywać je przez bezpiecznie zarządzane zmienne środowiskowe.

```
DATABASE_URL=...
OIDC_CLIENT_SECRET=...
SESSION_SECRET=...
STUDENT_INDEX_HMAC_KEY=...
```

Plik `.env` powinien być ignorowany przez Git, natomiast repozytorium może zawierać plik `.env.example` opisujący wymagane nazwy zmiennych bez wartości poufnych. Aplikacja powinna walidować konfigurację przy uruchamianiu i przerwać start, jeżeli brakuje wymaganych parametrów.

Testowanie

Kod powinien być objęty kilkoma poziomami testów:

- testy jednostkowe — funkcje punktowania, obliczanie czasu, walidacja;
- testy komponentów — formularze i nawigacja;
- testy integracyjne — API i baza danych;
- testy E2E — pełne podejście studenta;

- testy bezpieczeństwa — autoryzacja, XSS, CSRF, rate limiting;
- testy dostępności — klawiatura, czytnik ekranowy, automatyczne audyty;
- testy obciążeniowe — jednoczesne rozpoczęcie i zakończenie wejściówek;
- testy awarii — utrata sieci, ponowne wysłanie, wygaśnięcie sesji.

Kluczowe scenariusze E2E powinny obejmować:

1. Student rozpoczyna dozwolone podejście.
2. Odpowiedź zostaje automatycznie zapisana.
3. Odświeżenie strony odtwarza stan podejścia.
4. Utrata sieci nie powoduje utraty odpowiedzi.
5. Próba zapisu po upływie czasu jest odrzucana przez serwer.
6. Dwukrotne zakończenie podejścia nie tworzy dwóch wyników.
7. Student nie może odczytać wyników innego studenta.
8. Prowadzący widzi wyłącznie swoje kursy.
9. Dane telemetryczne nie zawierają wpisywanych znaków.

Kontrola jakości i proces wytwarzania

Każda zmiana powinna przechodzić przez przegląd kodu i automatyczny proces CI. Minimalny pipeline powinien wykonywać:

instalacja zależności

→ formatowanie

→ lint

→ kontrola typów

→ testy jednostkowe

→ testy integracyjne

→ budowanie aplikacji

→ skanowanie zależności

→ test migracji bazy

→ wdrożenie na środowisko testowe

Zmiany schematu bazy należy realizować przez wersjonowane migracje. Nie wolno ręcznie modyfikować produkcyjnej bazy bez odtworzonego skryptu migracyjnego. Funkcje o podwyższonym ryzyku, zwłaszcza monitoring, automatyczne ocenianie i eksport danych, powinny wymagać dodatkowego przeglądu bezpieczeństwa oraz ochrony danych.

Najważniejsza zasada projektowa

W omawianym systemie prawidłowo napisany kod WEB powinien realizować funkcje dydaktyczne przy możliwie małej ingerencji w prywatność studenta. Frontend odpowiada za ergonomiczny interfejs i zbieranie jawnie określonych metadanych, lecz nie może być źródłem prawdy w zakresie czasu, uprawnień i wyniku. Backend musi weryfikować każdą operację, a baza danych wymuszać integralność i separację dostępu. Kod powinien być semantyczny, modułowy, typowany, te-

stowany, dostępny cyfrowo i bezpieczny, natomiast monitoring — ograniczony, transparentny oraz technicznie odseparowany od mechanizmu wystawiania oceny.

Checklista wymagań dla platformy dydaktycznej

Status należy oznaczać jako:

- ☐ niespełnione
- ☒ częściowo spełnione
- ☐ spełnione
- N/D – nie dotyczy

1. Zakres funkcjonalny

- ☐ Strona umożliwia logowanie studentów i prowadzących.
- ☐ Student jest identyfikowany przez konto uczelniane lub numer indeksu.
- ☐ System weryfikuje poprawność numeru indeksu.
- ☐ Student widzi wyłącznie kursy, do których jest przypisany.
- ☐ Prowadzący widzi wyłącznie własne kursy i grupy.
- ☐ Strona umożliwia publikowanie materiałów dydaktycznych.
- ☐ Materiały można przypisać do kursu, grupy, tematu lub tygodnia.
- ☐ Materiały mogą mieć określoną datę publikacji i wycofania.
- ☐ System obsługuje pliki PDF, prezentacje, odnośniki i materiały tekstowe.
- ☐ Prowadzący może tworzyć, edytować, kopiować i usuwać wejściówki.
- ☐ Możliwe jest określenie czasu trwania wejściówki.
- ☐ Możliwe jest określenie okna dostępności wejściówki.
- ☐ Możliwe jest ograniczenie liczby podejść.
- ☐ Możliwe jest przypisanie wejściówki do konkretnej grupy.
- ☐ System obsługuje co najmniej pytania jednokrotnego i wielokrotnego wyboru.
- ☐ Kolejność pytań może być stała lub losowa.

- ☐ Kolejność odpowiedzi może być losowana.
- ☐ Punktacja jest konfigurowalna dla każdego pytania.
- ☐ Klucz poprawnych odpowiedzi nie jest przesyłany do przeglądarki studenta.
- ☐ Odpowiedzi są zapisywane automatycznie.
- ☐ Student otrzymuje informację o poprawnym zapisaniu odpowiedzi.
- ☐ Student może zakończyć próbę przed upływem czasu.
- ☐ System automatycznie kończy próbę po upływie limitu czasu.
- ☐ Po zakończeniu próby nie można zmieniać odpowiedzi.
- ☐ Wynik jest obliczany po stronie serwera.
- ☐ System zapisuje wynik, odpowiedzi, czas rozpoczęcia i zakończenia.
- ☐ Prowadzący może przeglądać wyniki pojedynczych studentów.
- ☐ Prowadzący może przeglądać zestawienia dla grupy.
- ☐ Wyniki można eksportować do CSV lub XLSX.
- ☐ Student ma dostęp do własnych wyników zgodnie z ustawieniami prowadzącego.

2. Obsługa podejścia i czasu

- ☐ Każda próba otrzymuje unikalny identyfikator.
- ☐ Czas rozpoczęcia jest ustalany przez serwer.
- ☐ Czas zakończenia jest ustalany przez serwer.
- ☐ Limit czasu nie zależy wyłącznie od zegara przeglądarki.
- ☐ Odświeżenie strony nie zeruje czasu wejściówki.
- ☐ Ponowne otwarcie strony przywraca aktualny stan próby.
- ☐ System zapisuje czas wyświetlania każdego pytania.
- ☐ System zapisuje czas udzielenia lub ostatniej zmiany odpowiedzi.
- ☐ System rejestruje liczbę zmian odpowiedzi.
- ☐ System rejestruje powroty do wcześniej odwiedzonych pytań.

- ☐ Czas bezczynności jest rozróżniany od aktywnej pracy.
- ☐ Próba zachowuje spójny status: utworzona, aktywna, zakończona, przerwana lub wygasła.
- ☐ Nie można utworzyć równolegle kilku aktywnych prób, jeżeli regulamin tego zabrania.
- ☐ Operacja zakończenia próby jest idempotentna.
- ☐ Ponowne wysłanie tego samego żądania nie tworzy duplikatu wyniku.

3. Monitoring aktywności

- ☐ Student przed rozpoczęciem próby otrzymuje informację o zakresie monitoringu.
- ☐ System rejestruje wyłącznie dane niezbędne do określonego celu.
- ☐ Rejestrowany jest czas aktywności w obrębie pytania.
- ☐ Rejestrowane są zmiany widoczności karty przeglądarki.
- ☐ Rejestrowana jest liczba opuszczeń karty lub okna.
- ☐ Aktywność wskaźnika jest próbkowana, a nie zapisywana dla każdego piksela ruchu.
- ☐ Częstotliwość próbkowania ruchu myszy jest ograniczona.
- ☐ Pozycje wskaźnika są zapisywane względnie, a nie jako dane umożliwiające odtworzenie pełnego pulpitu.
- ☐ Nie jest wykonywany pełny session replay.
- ☐ Nie są wykonywane zrzuty ekranu bez odrębnej podstawy i uzasadnienia.
- ☐ Nie jest rejestrowana treść wpisywanych znaków.
- ☐ System nie zapisuje haseł, tekstów wpisanych w innych polach ani danych schowka.
- ☐ Zdarzenia klawiatury są ograniczone do kategorii funkcjonalnych, np. nawigacja lub edycja.
- ☐ Monitoring nie traktuje korzystania wyłącznie z klawiatury jako zachowania podejrzanego.
- ☐ Telemetria jest wysyłana paczkami, a nie osobnym żądaniem dla każdego zdarzenia.
- ☐ Każde zdarzenie posiada numer sekwencyjny.

- ☐ Serwer wykrywa duplikaty i brakujące numery sekwencji.
- ☐ Telemetria ma określoną wersję schematu danych.
- ☐ Brak telemetrii nie powoduje automatycznego przypisania niesamodzielności.
- ☐ Zdarzenia behawioralne są jedynie materiałem pomocniczym dla prowadzącego.
- ☐ Ostateczna decyzja akademicka wymaga oceny człowieka.

4. Interfejs studenta

- ☐ Interfejs jasno wskazuje nazwę kursu i wejściówki.
- ☐ Widoczny jest numer aktualnego pytania i liczba wszystkich pytań.
- ☐ Widoczny jest pozostały czas.
- ☐ Widoczny jest status połączenia z serwerem.
- ☐ Widoczny jest status zapisu odpowiedzi.
- ☐ Pytania odpowiedziane, nieodpowiedziane i aktualnie otwarte są jednoznacznie oznaczone.
- ☐ Przyciski „Poprzednie”, „Następne” i „Zakończ” mają jednoznaczne etykiety.
- ☐ Zakończenie próby wymaga potwierdzenia.
- ☐ Komunikaty błędów wskazują przyczynę i sposób postępowania.
- ☐ Utrata połączenia nie powoduje utraty już zapisanych odpowiedzi.
- ☐ Po odzyskaniu połączenia dane są synchronizowane.
- ☐ Student otrzymuje potwierdzenie zakończenia próby.
- ☐ Potwierdzenie zawiera datę, czas i identyfikator próby.
- ☐ Interfejs nie zawiera elementów rozpraszaających.
- ☐ Podczas wejściówki nie są wyświetlane reklamy ani niepowiązane komunikaty.
- ☐ Układ działa poprawnie na komputerze, tablecie i telefonie.
- ☐ Elementy interaktywne nie są zbyt małe.
- ☐ Najważniejsze informacje pozostają widoczne przy powiększeniu strony.

5. Interfejs prowadzącego

- ☐ Prowadzący może utworzyć kurs i grupy.
- ☐ Prowadzący może przypisać studentów do grupy.
- ☐ Możliwy jest import listy studentów.
- ☐ Możliwa jest edycja materiałów i wejściówek bez ingerencji programisty.
- ☐ Dostępny jest podgląd wejściówki przed publikacją.
- ☐ System ostrzega o braku poprawnej odpowiedzi lub punktacji.
- ☐ System uniemożliwia publikację niekompletnej wejściówki.
- ☐ Prowadzący może ustawić próg zaliczenia.
- ☐ Prowadzący może ustawić sposób prezentacji wyników studentowi.
- ☐ Panel prezentuje liczbę rozpoczętych i zakończonych prób.
- ☐ Panel prezentuje średni wynik, medianę i rozkład wyników.
- ☐ Panel prezentuje czas odpowiedzi na poszczególne pytania.
- ☐ Panel wskazuje pytania o największej liczbie błędów.
- ☐ Panel umożliwia otwarcie szczegółów pojedynczej próby.
- ☐ Logi są prezentowane chronologicznie.
- ☐ Zdarzenia telemetryczne są opisane zrozumiałymi etykietami.
- ☐ System odróżnia logi techniczne od aktywności studenta.
- ☐ Dostęp do surowych logów jest ograniczony do uprawnionych osób.
- ☐ Każde wyświetlenie lub eksport danych studenta jest rejestrowane.
- ☐ Prowadzący może dodać komentarz do ręcznej analizy próby.
- ☐ System nie oznacza automatycznie studenta jako oszukującego.

6. Dostępność cyfrowa

- ☐ Strona spełnia wymagania WCAG 2.2 na poziomie AA.
- ☐ Cała funkcjonalność jest dostępna z klawiatury.

- ☐ Kolejność przechodzenia klawiszem Tab jest logiczna.
- ☐ Fokus klawiatury jest wyraźnie widoczny.
- ☐ Nie występują pułapki fokusu.
- ☐ Każde pole formularza ma powiązaną etykietę.
- ☐ Grupy odpowiedzi mają poprawne elementy fieldset i legend.
- ☐ Przyciski są tworzone jako elementy button, a nie nieinteraktywne div.
- ☐ Nagłówki mają logiczną hierarchię.
- ☐ Strona wykorzystuje semantyczne elementy HTML.
- ☐ Komunikaty dynamiczne są dostępne dla czytników ekranu.
- ☐ Zmiana statusu zapisu jest przekazywana przez aria-live.
- ☐ Kolor nie jest jedynym sposobem przekazywania informacji.
- ☐ Kontrast tekstu i elementów sterujących jest wystarczający.
- ☐ Interfejs działa przy powiększeniu co najmniej 200%.
- ☐ Treść nie wymaga przewijania w dwóch kierunkach przy typowym powiększeniu.
- ☐ Grafiki i ikony mają tekst alternatywny, gdy przekazują informację.
- ☐ Elementy dekoracyjne są ukryte przed technologiami asystującymi.
- ☐ Timer nie jest jedynym źródłem informacji o kończącym się czasie.
- ☐ Możliwe jest zastosowanie indywidualnego wydłużenia czasu.
- ☐ Dostępna jest procedura alternatywna dla studentów wymagających dostosowań.

7. Poprawność HTML, CSS i JavaScript

- ☐ Dokument zawiera poprawny doctype.
- ☐ Określono prawidłowy język strony.
- ☐ Ustawiono kodowanie UTF-8.
- ☐ Strona posiada unikalny i opisowy tytuł.
- ☐ HTML przechodzi walidację bez błędów krytycznych.

- ☐ Identyfikatory elementów są unikalne.
- ☐ Nie występuje zagnieżdżanie elementów niezgodne ze specyfikacją HTML.
- ☐ Style nie są mieszane z logiką biznesową.
- ☐ Kod JavaScript lub TypeScript nie jest umieszczany bezpośrednio w atrybutach HTML.
- ☐ Stosowany jest TypeScript w trybie ścisłym.
- ☐ Nie używa się `any` bez uzasadnienia.
- ☐ Funkcje mają jednoznaczne nazwy.
- ☐ Nazwy zmiennych opisują ich znaczenie.
- ☐ Komponenty mają ograniczoną odpowiedzialność.
- ☐ Logika komunikacji z API jest oddzielona od komponentów prezentacyjnych.
- ☐ Mechanizmy monitoringu są oddzielone od logiki oceniania.
- ☐ Procedury obsługi zdarzeń są usuwane podczas demontażu komponentu.
- ☐ Nie pozostają aktywne zbędne timery i obserwatory.
- ☐ Obsłużono błędy obietnic i żądań asynchronicznych.
- ☐ Nie występują niekontrolowane wycieki pamięci.
- ☐ Kod przechodzi kontrolę ESLint.
- ☐ Kod jest formatowany automatycznie, np. przez Prettier.
- ☐ W repozytorium nie występują sekrety ani hasła.
- ☐ Konfiguracja środowiskowa jest pobierana ze zmiennych środowiskowych.
- ☐ Plik `.env` jest wyłączony z repozytorium.
- ☐ Zależności są regularnie aktualizowane i skanowane pod kątem podatności.

8. API i backend

- ☐ API ma jednoznacznie zdefiniowane endpointy.
- ☐ API jest opisane w OpenAPI.
- ☐ Każde wejście jest walidowane po stronie serwera.

- ☐ Walidowane są typy, długości, zakresy i formaty danych.
- ☐ Serwer nie ufa identyfikatorom użytkownika przesłanym przez klienta.
- ☐ Właściciel próby jest ustalany na podstawie uwierzytelnionej sesji.
- ☐ Autoryzacja jest wykonywana dla każdego chronionego zasobu.
- ☐ Student nie może pobrać danych innego studenta.
- ☐ Prowadzący nie może pobrać danych z obcego kursu.
- ☐ Klucz odpowiedzi nigdy nie jest zwracany studentowi.
- ☐ Punktacja jest obliczana po stronie serwera.
- ☐ Okno czasowe wejściówki jest sprawdzane po stronie serwera.
- ☐ Liczba podejść jest kontrolowana po stronie serwera.
- ☐ Zapis odpowiedzi obsługuje kontrolę wersji lub rewizji.
- ☐ Konflikty zapisu są wykrywane.
- ☐ Żądania mają limity wielkości.
- ☐ Paczki telemetrii mają limit liczby zdarzeń.
- ☐ API stosuje rate limiting.
- ☐ Błędy API nie ujawniają stosu wywołań ani danych technicznych.
- ☐ Stosowane są właściwe kody HTTP.
- ☐ Operacje krytyczne są transakcyjne.
- ☐ Zakończenie próby i obliczenie wyniku nie mogą pozostawić stanu częściowego.

9. Baza danych

- ☐ Model danych rozdziela użytkowników, kursy, wejściówki, próby, odpowiedzi i telemetrię.
- ☐ Klucze główne są jednoznaczne.
- ☐ Klucze obce zapewniają spójność referencyjną.
- ☐ Istnieją ograniczenia unikalności dla numeru podejścia.

- ☐ Istnieje ograniczenie jednej odpowiedzi na pytanie w danej próbie.
- ☐ Stosowane są indeksy dla najczęstszych zapytań.
- ☐ Dane telemetryczne są oddzielone od danych wynikowych.
- ☐ Tabele telemetryczne mogą być partycjonowane według daty.
- ☐ Dane posiadają znaczniki utworzenia i modyfikacji.
- ☐ Daty są zapisywane w ujednoliconym formacie i strefie czasowej.
- ☐ Numer indeksu nie jest przechowywany w jawnej postaci bez uzasadnienia.
- ☐ Do wyszukiwania numeru indeksu stosowany jest bezpieczny mechanizm skrótu lub HMAC.
- ☐ Dostęp do rekordów jest ograniczony według roli i kursu.
- ☐ Wdrożono polityki Row-Level Security lub równoważny mechanizm.
- ☐ Usuwanie danych respektuje zależności między tabelami.
- ☐ Procedura retencji działa automatycznie.
- ☐ Kopie zapasowe obejmują dane i schemat.
- ☐ Procedura odtworzenia bazy została przetestowana.

10. Uwierzytelnianie i autoryzacja

- ☐ Preferowane jest logowanie przez uczelniane SSO.
- ☐ Integracja wykorzystuje standard OIDC lub SAML.
- ☐ Sesja użytkownika ma ograniczony czas ważności.
- ☐ Tokeny są weryfikowane po stronie serwera.
- ☐ Ciasteczka sesyjne mają ustawienia `HttpOnly`, `Secure` i właściwe `SameSite`.
- ☐ Identyfikatory sesji nie są przechowywane w adresie URL.
- ☐ System chroni operacje przed CSRF.
- ☐ Wylogowanie unieważnia sesję.
- ☐ Konta nieaktywne mogą zostać zablokowane.

- ☐ Role studenta, prowadzącego i administratora są rozdzielone.
- ☐ Uprawnienia są nadawane zgodnie z zasadą najmniejszych uprawnień.
- ☐ Administrator techniczny nie otrzymuje automatycznie dostępu do treści wyników.
- ☐ Operacje administracyjne są rejestrowane.
- ☐ Konta lokalne, jeżeli występują, używają bezpiecznego hashowania haseł.
- ☐ Dostęp administracyjny wymaga uwierzytelniania wieloskładnikowego.

11. Bezpieczeństwo aplikacji

- ☐ Cały ruch odbywa się przez HTTPS.
- ☐ Wymuszono aktualne wersje TLS.
- ☐ Aktywny jest nagłówek HSTS.
- ☐ Zdefiniowano Content Security Policy.
- ☐ Ustawiono X-Content-Type-Options: nosniff.
- ☐ Ograniczono możliwość osadzania strony w obcych ramkach.
- ☐ Ustawiono właściwą politykę Referrer-Policy.
- ☐ Dane wyświetlane w HTML są kodowane kontekstowo.
- ☐ Treści użytkownika są oczyszczane przed renderowaniem.
- ☐ Zapytania do bazy są parametryzowane.
- ☐ System jest odporny na SQL Injection.
- ☐ System jest odporny na XSS.
- ☐ System jest odporny na CSRF.
- ☐ Kontrolowany jest dostęp do przesyłanych plików.
- ☐ Typ i rozmiar pliku są walidowane.
- ☐ Nazwa pliku nie jest używana bezpośrednio jako ścieżka systemowa.
- ☐ Pliki przechowywane są poza katalogiem wykonywalnym aplikacji.
- ☐ Włączono skanowanie zależności.

- ☐ Wykonano testy bezpieczeństwa przed wdrożeniem.
- ☐ Wykonano testy kontroli dostępu między użytkownikami.
- ☐ Zdefiniowano procedurę reagowania na incydenty.
- ☐ Zdefiniowano procedurę zgłaszania naruszenia danych.
- ☐ Logi bezpieczeństwa są chronione przed modyfikacją.

12. Prywatność i RODO

- ☐ Określono administratora danych.
- ☐ Określono cele przetwarzania.
- ☐ Określono kategorie przetwarzanych danych.
- ☐ Określono właściwą podstawę prawną.
- ☐ Podstawa prawna została zweryfikowana przez IOD.
- ☐ Zgoda nie jest stosowana jako podstawa dla obowiązkowego monitoringu, jeżeli student nie może jej swobodnie odmówić.
- ☐ Przygotowano klauzulę informacyjną.
- ☐ Klauzula opisuje zakres monitoringu.
- ☐ Klauzula opisuje okres przechowywania danych.
- ☐ Klauzula opisuje odbiorców danych.
- ☐ Klauzula opisuje prawa studenta.
- ☐ Przeprowadzono ocenę skutków dla ochrony danych – DPIA.
- ☐ DPIA obejmuje monitoring zachowania studentów.
- ☐ DPIA obejmuje ryzyko błędnej interpretacji telemetrii.
- ☐ DPIA obejmuje ryzyko nieuprawnionego dostępu.
- ☐ Prowadzony jest rejestr czynności przetwarzania.
- ☐ Zdefiniowano politykę retencji.
- ☐ Surowa telemetria ma krótszą retencję niż wyniki.

- ☐ Dane są automatycznie usuwane po upływie retencji.
- ☐ Dane używane do analiz długoterminowych są anonimizowane.
- ☐ Pseudonimizacja nie jest błędnie traktowana jako anonimizacja.
- ☐ Student może uzyskać dostęp do swoich danych.
- ☐ Istnieje procedura sprostowania danych.
- ☐ Istnieje procedura ograniczenia przetwarzania lub usunięcia, gdy ma zastosowanie.
- ☐ Dostawcy hostingu mają odpowiednie umowy powierzenia.
- ☐ Dane są przechowywane w EOG lub wdrożono odpowiedni mechanizm transferowy.
- ☐ Zdefiniowano listę podmiotów przetwarzających i podprocesorów.
- ☐ Dane nie są wykorzystywane do nowych celów bez ponownej oceny zgodności.

13. Wydajność i niezawodność

- ☐ System został przetestowany dla zakładanej liczby równoczesnych studentów.
- ☐ Zapis odpowiedzi działa stabilnie przy obciążeniu szczytowym.
- ☐ Telemetria nie blokuje interfejsu wejściówki.
- ☐ Ruch wskaźnika jest ograniczany przez throttling.
- ☐ Częste operacje formularza są obsługiwane bez nadmiernego debounce.
- ☐ Telemetria jest buforowana lokalnie przy chwilowej utracie sieci.
- ☐ Bufor ma ograniczoną wielkość.
- ☐ System nie zużywa nadmiernie pamięci przeglądarki.
- ☐ Statyczne zasoby są kompresowane.
- ☐ Stosowane jest cache'owanie bez ujawniania danych prywatnych.
- ☐ Kod aplikacji jest dzielony na moduły ładowane według potrzeb.
- ☐ Duże pliki materiałów nie są przesyłane przez proces API bez potrzeby.
- ☐ System posiada monitoring czasu odpowiedzi i błędów.
- ☐ Zdefiniowano alarmy dla wzrostu liczby błędów.

- ☐ Zdefiniowano RPO i RTO.
- ☐ Kopie zapasowe są wykonywane automatycznie.
- ☐ Odtwarzanie z kopii zapasowej zostało przećwiczone.
- ☐ Awaria jednego komponentu nie powoduje utraty zapisanych odpowiedzi.

14. Testy

- ☐ Istnieją testy jednostkowe logiki oceniania.
- ☐ Istnieją testy jednostkowe limitów czasu.
- ☐ Istnieją testy walidacji danych.
- ☐ Istnieją testy integracyjne API i bazy danych.
- ☐ Istnieją testy autoryzacji dla każdej roli.
- ☐ Istnieją testy dostępu między różnymi kursami.
- ☐ Istnieją testy rozpoczęcia, wznowienia i zakończenia próby.
- ☐ Istnieją testy automatycznego zapisu odpowiedzi.
- ☐ Istnieją testy konfliktu równoczesnego zapisu.
- ☐ Istnieją testy utraty i odzyskania połączenia.
- ☐ Istnieją testy wygaśnięcia czasu.
- ☐ Istnieją testy podwójnego kliknięcia „Zakończ”.
- ☐ Istnieją testy duplikatów telemetry.
- ☐ Istnieją testy niepoprawnych paczek telemetry.
- ☐ Istnieją testy obciążeniowe.
- ☐ Istnieją testy bezpieczeństwa.
- ☐ Istnieją testy dostępności automatycznej.
- ☐ Przeprowadzono ręczny test obsługi wyłącznie klawiaturą.
- ☐ Przeprowadzono test z czytnikiem ekranu.
- ☐ Przeprowadzono test na aktualnych wersjach głównych przeglądarek.

- ☐ Przeprowadzono test na urządzeniach mobilnych.
- ☐ Przeprowadzono pilotaż z rzeczywistą grupą studentów.

15. Repozytorium i jakość procesu wytwarzania

- ☐ Kod znajduje się w systemie kontroli wersji.
- ☐ Repozytorium posiada czytelny plik README.
- ☐ README zawiera instrukcję uruchomienia.
- ☐ README opisuje architekturę systemu.
- ☐ README opisuje wymagane zmienne środowiskowe.
- ☐ Repozytorium zawiera przykładowy plik `.env.example`.
- ☐ Zdefiniowano konwencję nazewnictwa gałęzi i commitów.
- ☐ Zmiany przechodzą przegląd kodu.
- ☐ Gałąź produkcyjna jest chroniona.
- ☐ Pipeline CI uruchamia linting.
- ☐ Pipeline CI uruchamia testy.
- ☐ Pipeline CI wykonuje build aplikacji.
- ☐ Pipeline CI wykonuje skan podatności zależności.
- ☐ Migracje bazy są wersjonowane.
- ☐ Migracje są testowane przed wdrożeniem.
- ☐ Wdrożenie może zostać wycofane.
- ☐ Środowisko testowe jest oddzielone od produkcyjnego.
- ☐ Dane produkcyjne nie są kopiowane do środowiska testowego bez anonimizacji.
- ☐ Wersja aplikacji jest widoczna w logach i diagnostyce.
- ☐ Każde wdrożenie posiada opis zmian.

16. Kryteria odbioru produkcyjnego

Strona może zostać uznana za gotową do wdrożenia, gdy:

- ☐ Wszystkie wymagania krytyczne dotyczące bezpieczeństwa są spełnione.
- ☐ Wszystkie wymagania krytyczne dotyczące prywatności są spełnione.
- ☐ DPIA została zatwierdzona.
- ☐ Klauzula informacyjna została opublikowana.
- ☐ Nie są rejestrowane treści naciskanych klawiszy.
- ☐ Klucz odpowiedzi nie jest dostępny po stronie klienta.
- ☐ Wynik i czas są ustalane po stronie serwera.
- ☐ Autoryzacja per kurs i per użytkownik została przetestowana.
- ☐ System poprawnie działa po utracie i odzyskaniu połączenia.
- ☐ Wyniki nie są tracone przy odświeżeniu strony.
- ☐ System spełnia WCAG 2.2 AA w zakresie krytycznych ścieżek.
- ☐ Wykonano testy obciążeniowe dla zakładanej liczby użytkowników.
- ☐ Kopie zapasowe i odtwarzanie zostały zweryfikowane.
- ☐ Monitoring i alerty produkcyjne są aktywne.
- ☐ Prowadzący i administratorzy zostali przeszkoleni.
- ☐ Przeprowadzono pilotaż i usunięto błędy krytyczne.
- ☐ Zatwierdzono regulamin korzystania z systemu.
- ☐ Zdefiniowano procedurę odwołania od wyniku lub analizy incydentu.
- ☐ Wyznaczono właściciela biznesowego i technicznego systemu.

Minimalny zestaw krytyczny

Bez spełnienia poniższych punktów system nie powinien obsługiwać rzeczywistych wejściówek:

- ☐ Bezpieczne uwierzytelnianie i autoryzacja.
- ☐ Serwerowa kontrola czasu i liczby podejść.
- ☐ Serwerowe obliczanie wyników.
- ☐ Automatyczny i odporny zapis odpowiedzi.

- ☐ Brak klucza odpowiedzi w kodzie klienta.
- ☐ Brak zapisu treści naciskanych klawiszy.
- ☐ Jasna informacja o monitoringu.
- ☐ DPIA i konsultacja z IOD.
- ☐ Szyfrowanie transmisji i danych.
- ☐ Ograniczony dostęp prowadzącego do własnych kursów.
- ☐ Polityka retencji i automatycznego usuwania logów.
- ☐ Testy dostępności, bezpieczeństwa i obciążenia.
- ☐ Kopie zapasowe oraz przetestowane odtwarzanie.
- ☐ Ręczna ocena wszelkich flag dotyczących niesamodzielności.

Literatura

[1] [33] Art. 35 GDPR – Data protection impact assessment - General Data Protection Regulation (GDPR) <https://gdpr-info.eu/art-35-gdpr/>

[2] [4] [6] [12] [32] Art. 6 GDPR – Lawfulness of processing - General Data Protection Regulation (GDPR) <https://gdpr-info.eu/art-6-gdpr/>

[3] [19] Next.js Docs: App Router <https://nextjs.org/docs/app>

[5] edpb.europa.eu https://www.edpb.europa.eu/system/files/2026-02/edpb-summary-gdpr-data-protection-design-default_en.pdf

[7] [26] Art. 12 GDPR – Transparent information, communication and modalities for the exercise of the rights of the data subject - General Data Protection Regulation (GDPR) <https://gdpr-info.eu/art-12-gdpr/>

[8] Web Content Accessibility Guidelines (WCAG) 2.2 <https://www.w3.org/TR/WCAG22>

[9] Art. 6 GDPR – Lawfulness of processing - General Data <https://gdpr-info.eu/art-6-gdpr>

[10] [11] Guidelines 05/2020 https://dpa.gr/sites/default/files/2020-07/edpb_guidelines_202005_consent_en.pdf

[13] Data protection basics https://www.edpb.europa.eu/sme-data-protection-guide/data-protection-basics_en

[14] Art. 15 GDPR – Right of access by the data subject - General Data Protection Regulation (GDPR) <https://gdpr-info.eu/art-15-gdpr/>

- [15] Anonymisation/pseudonymisation
https://www.edpb.europa.eu/topics/ai-and-technology/anonymisationpseudonymisation_en
- [16] Art. 32 GDPR – Security of processing - General Data Protection Regulation (GDPR)
<https://gdpr-info.eu/art-32-gdpr/>
- [17] Art. 22 GDPR – Automated individual decision-making, including profiling - General Data Protection Regulation (GDPR) <https://gdpr-info.eu/art-22-gdpr/>
- [18] Documentation | NestJS - A progressive Node.js framework <https://docs.nestjs.com>
- [20] Pointer events - Web APIs | MDN
https://developer.mozilla.org/en-US/docs/Web/API/Pointer_events
- [21] PostgreSQL: Documentation: 18: 32.19. SSL Support
<https://www.postgresql.org/docs/current/libpq-ssl.html>
- [22] Documentation: 18: 5.9. Row Security Policies <https://www.postgresql.org/docs/current/ddl-rowsecurity.html>
- [23] Art. 28 GDPR – Processor - General Data Protection Regulation (GDPR)
<https://gdpr-info.eu/art-28-gdpr/>
- [24] Documentation: 18: 8.14. JSON Types <https://www.postgresql.org/docs/current/datatype-json.html>
- [25] Mermaid | Diagramming and charting tool <https://mermaid.js.org>
- [27] OpenAPI Initiative – The OpenAPI Initiative provides an open source
<https://www.openapis.org>
- [28] Logging Cheat Sheet https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
- [29] Document: visibilitychange event - Web APIs | MDN
https://developer.mozilla.org/en-US/docs/Web/API/Document/visibilitychange_event
- [30] Cost-Optimized: Cheap Entry Level <https://www.hetzner.com/cloud/cost-optimized>
- [31] DPA <https://supabase.com/legal/dpa>